DESENVOLVIMENTO FRONT-END · MARCO 5

VueNautas

Gerenciador de tarefas e projetos com Vue.js e TypeScript

Thiago Henrique

Eduarda

Corina



O que evoluiu neste marco

A aplicação passou a explicar melhor as operações e os estados do sistema.

ISSUES #77 E #81

Operações seguras

Excluir com confirmação.
Informar falhas ao salvar e excluir.

ISSUES #78, #79 E #80

Estados da interface

Mostrar carregamento e erro.
Permitir nova tentativa e explicar estados vazios.

ISSUES #82 E #90

Qualidade e registros

Tratar registros inexistentes.
Auditar acessibilidade e responsividade.

PADRÃO DO CÓDIGO **Composables reutilizam funções + componentes reutilizam a interface.**

A exclusão começa pela confirmação

A função compartilhada removerTarefa fica no composable useTarefas.

USETAREFAS.TS · INÍCIO DE REMOVERTAREFA()

```
async function removerTarefa(id: string) {
  const tarefa = tarefas.value.find(
    (item) => item.id === id,
  )
  if (!tarefa) return false

  const confirmado = window.confirm(
    `Excluir "${tarefa.titulo}"?`
  )
  if (!confirmado) return false
```

O QUE A FUNÇÃO FAZ

- **Localiza**
  find procura a tarefa pelo ID.
- **Confirma**
  window.confirm pergunta antes de excluir.
- **Interrompe**
  Se não encontrar ou se cancelar, retorna false.

A mesma função atende lista e quadro.

A API remove; a lista acompanha

Depois da confirmação, o composable atualiza o estado compartilhado.

USETAREFAS.TS · EXECUÇÃO E FINALIZAÇÃO

```
excluindoId.value = id
try {
  await excluirTarefaApi(id)
  tarefas.value = tarefas.value.filter(
    (item) => item.id !== id,
  )
  return true
} catch (erro) { ... }
finally { excluindoId.value = null }
```

POR QUE FUNCIONA

- **DELETE**
  excluirTarefaApi envia a solicitação à API.
- **filter()**
  Cria uma lista sem a tarefa removida.
- **finally**
  Libera o estado de exclusão mesmo se ocorrer falha.

O cartão recebe o evento e chama removerTarefa.

A camada da API padroniza os erros

A classe ErroApi guarda o status HTTP; requisitar<T>() centraliza a verificação.

API.TS · VERIFICAÇÃO DA RESPOSTA HTTP

```
if (!resposta.ok) {
  throw new ErroApi(
    resposta.status,
    mensagemPorStatus(resposta.status),
  )
}
```

CONEXÃO
Sem resposta da API

STATUS HTTP
Erro identificado

TELA
Mensagem compreensível

O formulário avisa sem apagar o que foi digitado

O catch apresenta a mensagem; os campos continuam nas variáveis reativas.

FORMULARIOTAREFA.VUE · CATCH DE SALVARTAREFA()

```
catch (erroSalvar) {
  const mensagem = erroSalvar instanceof Error
    ? erroSalvar.message
    : 'Verifique se a API está ativa.'

  erro.value =
    `Não foi possível salvar a tarefa. ${mensagem}`
}
```

O QUE A PESSOA PERCEBE

- **A operação falhou**
  A API não confirmou o cadastro ou a edição.
- **O erro aparece**
  A mensagem é apresentada na própria tela.
- **Os dados permanecem**
  O formulário não limpa o que a pessoa digitou.

O mesmo padrão foi aplicado ao formulário de projeto.

A espera também é um estado da interface

EstadoConsulta é um componente reutilizado nas telas que buscam dados.

ESTADOCONSULTA.VUE · CARREGAMENTO E SLOT

```
<p v-if="carregando" role="status">
  {{ mensagemCarregamento ||
     'Carregando...' }}
</p>


<slot v-else />
```

COMO FUNCIONA

- **A tela informa**
  Envia carregando e o texto que deseja exibir.
- **O componente mostra**
  v-if exibe uma mensagem de espera padronizada.
- **O conteúdo retorna**
  slot libera a lista ou os detalhes da tela.

role="status" ajuda a comunicar a atualização.

A mensagem permite tentar novamente

O componente emite um evento; a tela executa sua própria função de busca.

COMPONENTE · EVENTO EMITIDO E RECEBIDO

```
const emit = defineEmits<{
  (evento: 'tentar-novamente'): void
}>()


emit('tentar-novamente')


@tentar-novamente="buscarTarefas"
```

COMUNICAÇÃO

- **defineEmits**
  Declara o evento que o componente pode emitir.
- **Clique**
  O botão dispara tentar-novamente.
- **A tela responde**
  buscarTarefas executa a consulta novamente.

A consulta não é duplicada dentro do componente.

Três situações, três mensagens diferentes

A interface explica por que nenhum cartão está sendo exibido.

LISTA ORIGINAL

Nenhuma tarefa cadastrada

```
tarefas.length === 0
```

O array original não possui tarefas.

BUSCA E FILTROS

Nenhuma tarefa encontrada

```
tarefasFiltradas.length === 0
```

Há tarefas, mas nenhuma corresponde aos filtros.

DETALHE DO PROJETO

Projeto sem tarefas vinculadas

```
tarefas.length === 0
```

O projeto existe, mas não tem tarefas relacionadas.

O botão “Limpar filtros” reaproveita storeFiltros.limpar().

O 404 informa quando o registro não existe

O mesmo tratamento foi aplicado aos detalhes da tarefa e do projeto.

DETALHETAREFA.VUE · IDENTIFICAR O 404

```
if (erroBusca instanceof ErroApi &&
    erroBusca.status === 404) {
  erro.value =
    `Nenhuma tarefa foi encontrada com o ID ${id}.`
  return
}

finally {
  if (numeroRequisicao === requisicaoAtual) {
    carregando.value = false
  }
}
```

FLUXO DO TRATAMENTO

- **Ler o ID**
  A tela recebe o identificador pela rota.
- **Consultar a API**
  buscarTarefaApi tenta localizar o registro.
- **Tratar 404**
  Exibe mensagem específica e encerra o carregamento.

DetalheProjeto.vue aplica a mesma regra.

O Lighthouse apontou 95 pontos em acessibilidade

O relatório foi gerado em emulação de Moto G Power.

95

/ 100

ACESSIBILIDADE

Lighthouse 13.3 · Moto G Power

ACHADO DO RELATÓRIO

Contraste insuficiente

O Lighthouse identificou texto e fundo com contraste abaixo do recomendado.

10 itens adicionais para conferência manual

A pontuação é uma referência automática; não substitui testes com teclado e tecnologias assistivas.

O CSS adapta a interface e destaca o foco

A auditoria considera tamanhos de tela e navegação sem mouse.

STYLE.CSS · ADAPTAÇÃO EM TELAS MENORES

```css
@media (max-width: 700px) {
  .grade-cartoes {
    grid-template-columns:
      minmax(0, 1fr);
  }
}

a:focus-visible, button:focus-visible {
  outline: 3px solid #174f87;
}
```

O QUE VERIFICAR

- **Teclado**
  Tab e Shift + Tab devem alcançar os controles.
- **Foco visível**
  O elemento ativo precisa ficar destacado.
- **Telas menores**
  Conferir navegação, filtros, formulários e quadro.

Referências de teste: 320 px · 375 px · 768 px.

Do comportamento ao código

Agora vamos observar as funcionalidades diretamente no sistema.

01 ISSUES #77 E #81
Operações seguras
Excluir uma tarefa e observar o tratamento de falhas.

02 ISSUES #78, #79 E #80
Estados da interface
Carregamento, erro com nova tentativa e estados vazios.

03 ISSUES #90 E #82
Registro e qualidade
Abrir rota inexistente e apresentar a auditoria.

ENCERRAMENTO

Mais clareza para quem usa.

Estados definidos e código mais reutilizável.

OBRIGADO!